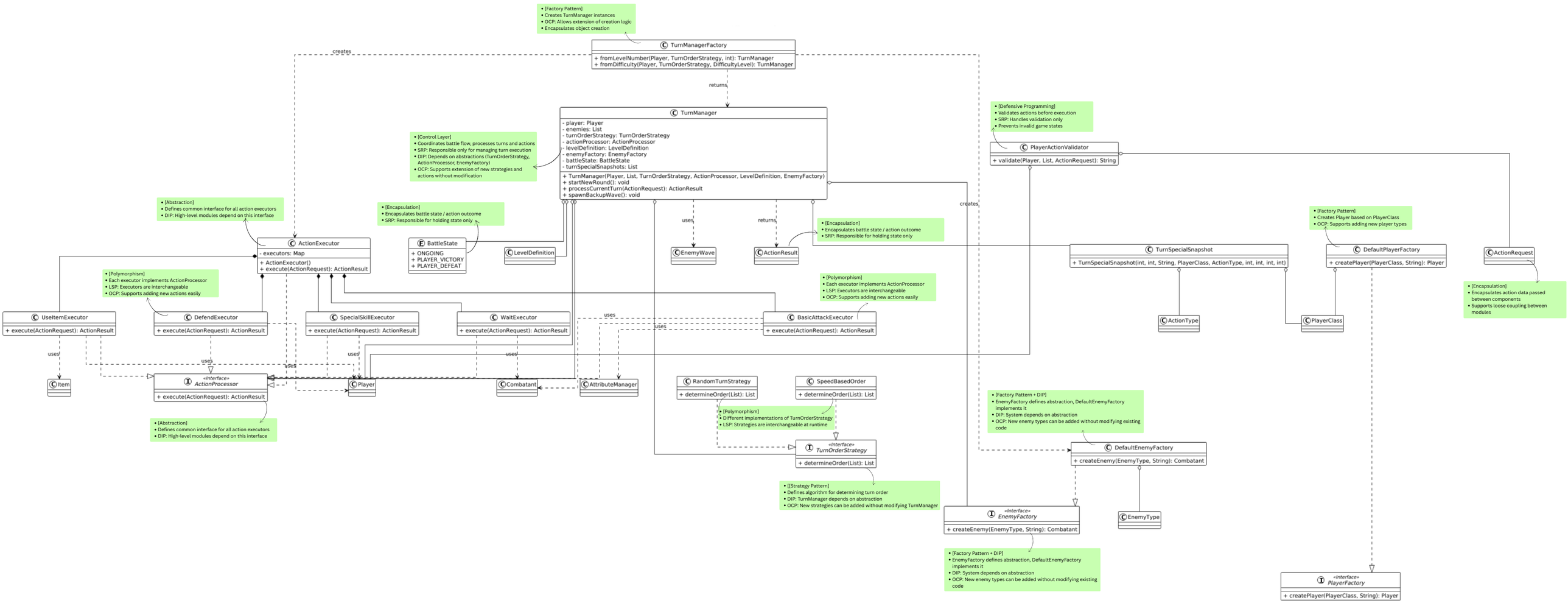
• [Factory Pattern]
• Creates TurnManager instances
• OCP: Allows extension of creation logic
• Encapsulates object creation
TurnManagerFactory
+ fromLevelNumber(Player, TurnOrderStrategy, int): TurnManager
+ fromDifficulty(Player, TurnOrderStrategy, DifficultyLevel): TurnManager
creates
returns
TurnManager
- player: Player
- enemies: List
- turnOrderStrategy: TurnOrderStrategy
- actionProcessor: ActionProcessor
- levelDefinition: LevelDefinition
- enemyFactory: EnemyFactory
- battleState: BattleState
- turnSpecialSnapshots: List
+ TurnManager(Player, List, TurnOrderStrategy, ActionProcessor, LevelDefinition, EnemyFactory)
+ startNewRound(): void
+ processCurrentTurn(ActionRequest): ActionResult
+ spawnBackupWave(): void
• [Control Layer]
• Coordinates battle flow, processes turns and actions
• SRP: Responsible only for managing turn execution
• DIP: Depends on abstractions (TurnOrderStrategy, ActionProcessor, EnemyFactory)
• OCP: Supports extension of new strategies and actions without modification
• [Defensive Programming]
• Validates actions before execution
• SRP: Handles validation only
• Prevents invalid game states
PlayerActionValidator
+ validate(Player, List, ActionRequest): String
• [Abstraction]
• Defines common interface for all action executors
• DIP: High-level modules depend on this interface
• [Encapsulation]
• Encapsulates battle state / action outcome
• SRP: Responsible for holding state only
ActionExecutor
- executors: Map
+ ActionExecutor()
+ execute(ActionRequest): ActionResult
BattleState
+ ONGOING
+ PLAYER_VICTORY
+ PLAYER_DEFEAT
LevelDefinition
uses
EnemyWave
returns
ActionResult
• [Encapsulation]
• Encapsulates battle state / action outcome
• SRP: Responsible for holding state only
creates
• [Factory Pattern]
• Creates Player based on PlayerClass
• OCP: Supports adding new player types
TurnSpecialSnapshot
+ TurnSpecialSnapshot(int, int, String, PlayerClass, ActionType, int, int, int, int)
DefaultPlayerFactory
+ createPlayer(PlayerClass, String): Player
ActionRequest
• [Encapsulation]
• Encapsulates action data passed between components
• Supports loose coupling between modules
• [Polymorphism]
• Each executor implements ActionProcessor
• LSP: Executors are interchangeable
• OCP: Supports adding new actions easily
• [Polymorphism]
• Each executor implements ActionProcessor
• LSP: Executors are interchangeable
• OCP: Supports adding new actions easily
UseItemExecutor
+ execute(ActionRequest): ActionResult
DefendExecutor
+ execute(ActionRequest): ActionResult
SpecialSkillExecutor
+ execute(ActionRequest): ActionResult
WaitExecutor
+ execute(ActionRequest): ActionResult
BasicAttackExecutor
+ execute(ActionRequest): ActionResult
ActionType
PlayerClass
uses
Item
«Interface»
ActionProcessor
+ execute(ActionRequest): ActionResult
Player
Combatant
AttributeManager
RandomTurnStrategy
+ determineOrder(List): List
SpeedBasedOrder
+ determineOrder(List): List
• [Abstraction]
• Defines common interface for all action executors
• DIP: High-level modules depend on this interface
• [Polymorphism]
• Different implementations of TurnOrderStrategy
• LSP: Strategies are interchangeable at runtime
«Interface»
TurnOrderStrategy
+ determineOrder(List): List
• [Factory Pattern + DIP]
• EnemyFactory defines abstraction, DefaultEnemyFactory implements it
• DIP: System depends on abstraction
• OCP: New enemy types can be added without modifying existing code
DefaultEnemyFactory
+ createEnemy(EnemyType, String): Combatant
• [[Strategy Pattern]
• Defines algorithm for determining turn order
• DIP: TurnManager depends on abstraction
• OCP: New strategies can be added without modifying TurnManager
«Interface»
EnemyFactory
+ createEnemy(EnemyType, String): Combatant
EnemyType
• [Factory Pattern + DIP]
• EnemyFactory defines abstraction, DefaultEnemyFactory implements it
• DIP: System depends on abstraction
• OCP: New enemy types can be added without modifying existing code
«Interface»
PlayerFactory
+ createPlayer(PlayerClass, String): Player